

Day 2, Session 1: Thread-Based Parallelism

Duration: 2 hours (theory + live demos)

Prerequisites: Day 1 — threads, processes, GIL basics

Demos folder: [day2/demo/](#)

Table of Contents

1. [Threads Recap & Introspection](#)
 2. [Subclassing Thread](#)
 3. [Synchronization with Lock and RLock](#)
 4. [Synchronization with Semaphore](#)
 5. [Coordination with Condition](#)
 6. [Coordination with Event](#)
 7. [Coordination with Barrier](#)
 8. [Thread Communication using Queue](#)
-

1. Threads Recap & Introspection

Day 1 covered creating threads with `threading.Thread(target=..., args=...)`. Today we go deeper — starting with tools to **inspect** threads at runtime.

Thread Introspection Functions

Function	Returns
<code>threading.current_thread()</code>	The Thread object for the caller
<code>threading.main_thread()</code>	The main thread object
<code>threading.enumerate()</code>	List of all alive Thread objects
<code>threading.active_count()</code>	Number of currently alive threads

```
import threading

def worker():
    me = threading.current_thread()
    print(f"I am '{me.name}', alive={me.is_alive()}")
    print(f"Active threads: {threading.active_count()}")

t = threading.Thread(target=worker, name="MyWorker")
t.start()
t.join()
```

These are invaluable for **debugging** — if your program hangs, call `threading.enumerate()` to see which threads are still alive.

2. Subclassing Thread

Instead of passing a `target` function, you can **subclass** `threading.Thread` and override its `run()` method. This is useful when a thread needs to carry its own state.

```
class DownloadThread(threading.Thread):
    def __init__(self, url):
        super().__init__()
        self.url = url
        self.result = None

    def run(self):
        # This method runs in the new thread
        self.result = f"Downloaded {self.url}"

t = DownloadThread("https://example.com/data.csv")
t.start()
t.join()
print(t.result)    # "Downloaded https://example.com/data.csv"
```

When to use subclassing vs `target=`:

Approach	Best for
<code>target=</code>	Simple, stateless tasks
Subclass	Threads that carry state or have lifecycle

3. Synchronization with Lock and RLock

Lock (Recap)

A **Lock** ensures **mutual exclusion** — only one thread can hold it at a time. Any other thread that tries to acquire it will **block** until it's released.

```
lock = threading.Lock()

def safe_work():
    with lock:
        # acquire → do work → release (auto)
        # critical section — only one thread here at a time
        modify_shared_data()
```

RLock (Reentrant Lock)

A regular **Lock** will **deadlock** if the same thread tries to acquire it twice. An **RLock** solves this — the **same thread** can acquire it multiple times, as long as it releases it the same number of times.

Regular Lock:

```
Thread-1 acquires → OK
Thread-1 acquires → 
DEADLOCK!
```

RLock (Reentrant Lock):

```
Thread-1 acquires → OK
Thread-1 acquires → OK
(count = 2)
Thread-1 releases
(count = 1)
Thread-1 releases
(count = 0, UNLOCKED)
```

Use case: Recursive functions or nested method calls where both the outer and inner functions need the same lock:

```
rlock = threading.RLock()

def outer():
    with rlock:
        inner()           # won't deadlock – same thread

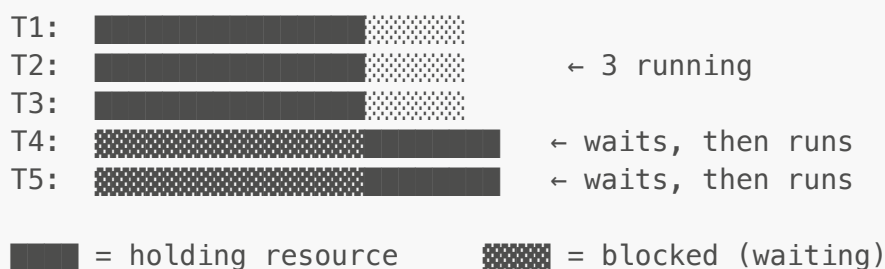
def inner():
    with rlock:           # acquires the same RLock again → OK
        do_work()
```

4. Synchronization with Semaphore

A **Semaphore** manages an internal counter that represents a **pool of resources**. It allows up to N threads to access a resource concurrently.

Semaphore(3) – allows max 3 concurrent threads

Time →



Method	Effect
<code>acquire()</code>	Decrement counter. If 0, block until > 0.
<code>release()</code>	Increment counter. Wake one blocked thread.

Real-world use cases:

- Database connection pools (max 10 connections)
- Rate limiting API calls (max 5 concurrent requests)
- Controlling access to limited hardware (max 2 GPUs)

```
pool = threading.Semaphore(3)    # max 3 concurrent

def use_resource(thread_id):
    with pool:                    # acquire on enter, release on exit
        print(f"Thread {thread_id} using resource")
        time.sleep(1)
```

Semaphore vs Lock:

Feature	Lock	Semaphore(N)
Max holders	1	N
Use case	Mutual exclusion	Resource pool
Special case	—	Semaphore(1) $\approx$ Lock

Demo Time

Now, let's see these theoretical topics in action.

- `01_thread_sync_primitives.py` — Three progressive demos:
 1. `demo_lock()` — Shared counter with Lock
 2. `demo_rlock()` — Recursive function with RLock (no deadlock)
 3. `demo_semaphore()` — Limit concurrent access to 3 threads

5. Coordination with Condition

A **Condition** variable lets threads **wait** for a specific state to become true, and another thread **notifies** them when it does. It wraps a **Lock** internally.

Producer–Consumer with Condition:

Consumer	Producer
_____	_____
acquire lock	

```

while not items:
    wait() ← releases lock
            and blocks
            acquire lock
            items.append(data)
            notify() → wakes consumer
            release lock
    ← wakes up, re-acquires lock
process items
release lock

```

```

└─ wake up!
└─ wake up!

```

Method	Effect
<code>set()</code>	Set flag to True , wake all waiters
<code>clear()</code>	Reset flag to False
<code>wait()</code>	Block until flag is True
<code>is_set()</code>	Check flag without blocking

Event vs Condition:

Feature	Event	Condition
Complexity	Simple flag (True/False)	Arbitrary condition
Waiters	All wake on set()	notify() wakes one/all
Use case	"Go!" signal, shutdown flag	Producer-consumer, state

7. Coordination with Barrier

A **Barrier** blocks threads until a **fixed number** have all arrived at the same point, then releases them all simultaneously.

Barrier(3) – requires 3 threads to proceed

Thread 1: ████ Phase 1 ████ → wait() → BLOCKED
 Thread 2: ████████ Phase 1 ████ → wait() → BLOCKED
 Thread 3: ████████████ Phase 1 ████ → wait()

All 3 arrived! —————

Thread 1: ← released → ████ Phase 2
 Thread 2: ← released → ████ Phase 2
 Thread 3: ← released → ████ Phase 2

Use case: Phased computation where all threads must complete phase N before any thread starts phase N+1 (e.g., parallel matrix algorithms, simulation time steps).

```

barrier = threading.Barrier(3)

def worker(thread_id):
    print(f"Thread {thread_id}: Phase 1 done")
    barrier.wait()      # block until all 3 arrive
    print(f"Thread {thread_id}: Phase 2 started")

```

8. Thread Communication using Queue

The `queue` module provides **thread-safe** data structures — no external locking needed.

Queue Types

Queue Type	Order	Use Case
<code>queue.Queue</code>	FIFO (First In, First Out)	Standard task queues
<code>queue.LifoQueue</code>	LIFO (Last In, First Out)	Stack-like behavior
<code>queue.PriorityQueue</code>	Lowest value first	Priority scheduling

PriorityQueue Example

Items are tuples of (`priority`, `data`). Lower priority number = higher priority:

```

import queue

pq = queue.PriorityQueue()
pq.put((3, "Low priority"))
pq.put((1, "URGENT"))
pq.put((2, "Normal"))

while not pq.empty():
    priority, task = pq.get()
    print(f"Processing: {task}")
# Output: URGENT → Normal → Low priority

```

Demo Time

Now, let's see these theoretical topics in action.

- `02_thread_coordination.py` — Four progressive demos:
 1. `demo_condition()` — Producer-consumer with Condition (wait/notify)
 2. `demo_event()` — Workers wait for a "go" signal
 3. `demo_barrier()` — Three threads synchronize between phases
 4. `demo_queue_types()` — PriorityQueue ordering

Session 1 Summary

Primitive	Purpose	Key Behavior
Lock	Mutual exclusion	Only one thread holds it at a time
RLock	Recursive locking	Same thread can acquire multiple times
Semaphore	Resource pool limiting	Up to N threads can hold it concurrently
Condition	State-based coordination	wait() / notify() for producer-consumer
Event	One-shot signaling	set() wakes all waiters simultaneously
Barrier	Phase synchronization	All threads must arrive before any proceed
Queue	Thread-safe communication	FIFO, LIFO, and Priority variants

Choosing the Right Primitive

Need mutual exclusion?

- └ Yes, simple → Lock
- └ Yes, recursive/nested → RLock
- └ Yes, but allow N concurrent → Semaphore

Need coordination?

- └ Wait for a condition → Condition
- └ Wait for a signal → Event
- └ Wait for all to arrive → Barrier

Need to pass data?

- └ Use Queue (FIFO / LIFO / Priority)

Break time! Session 2 covers Process-Based Parallelism → [README-session2.md](#)